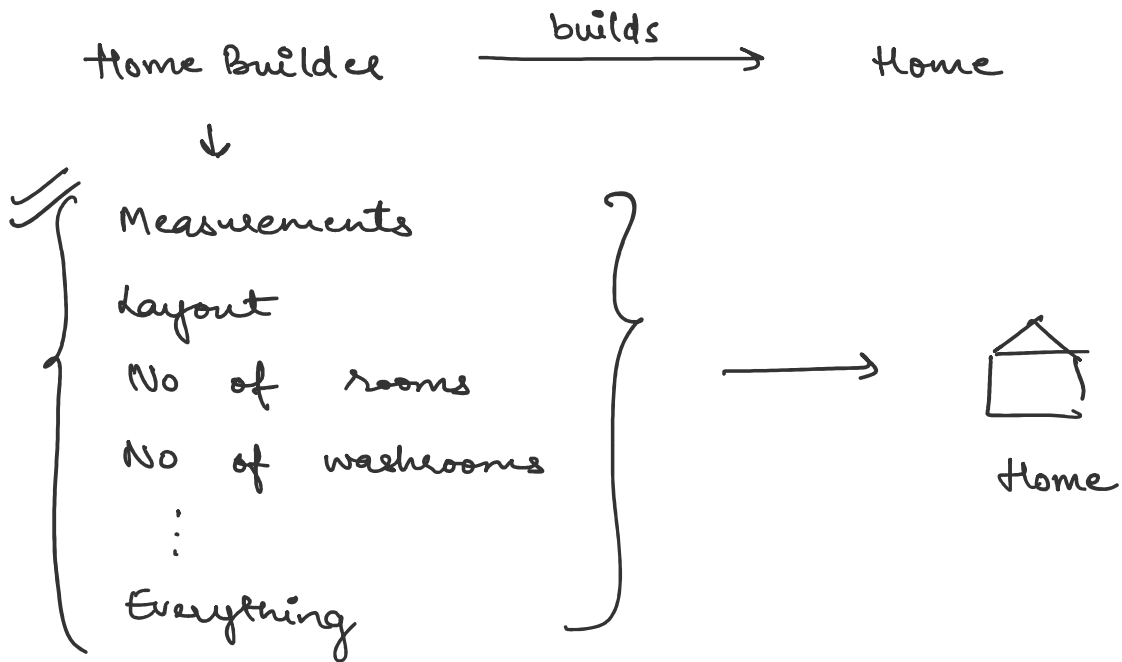
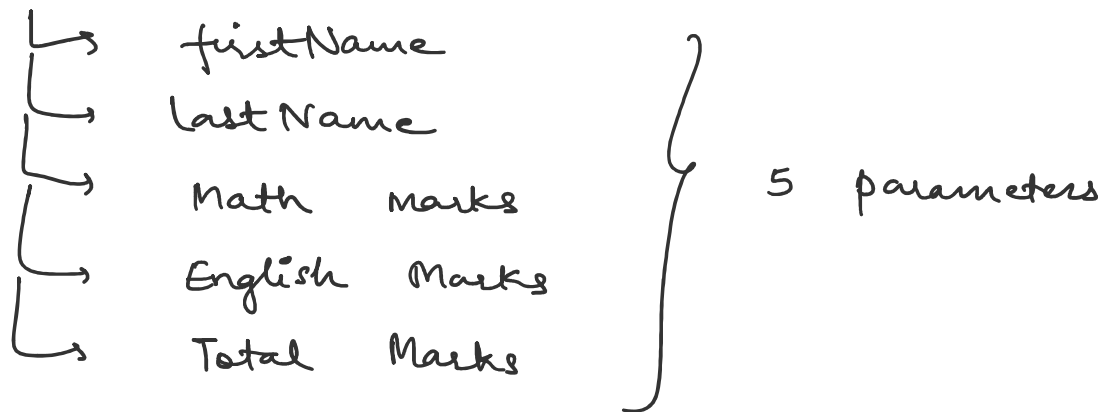


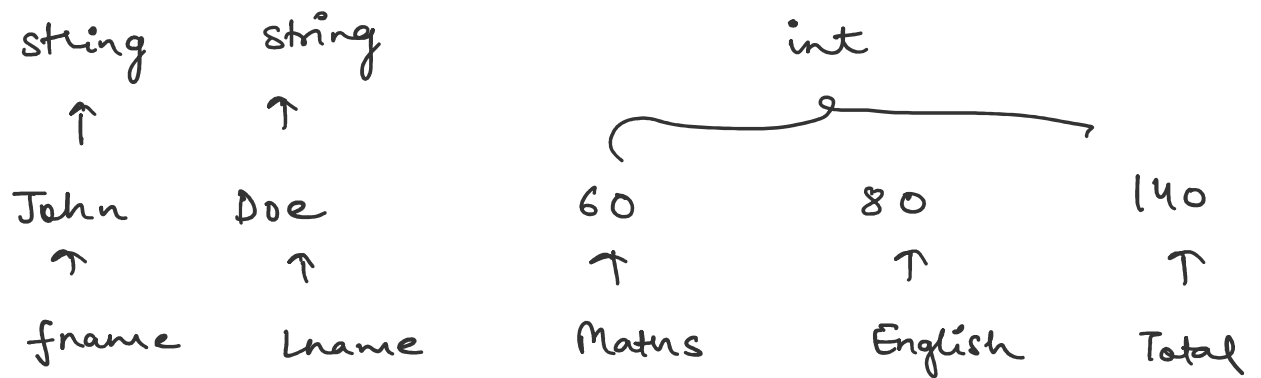
Builder Design Pattern



Student



Constructor of Student class -

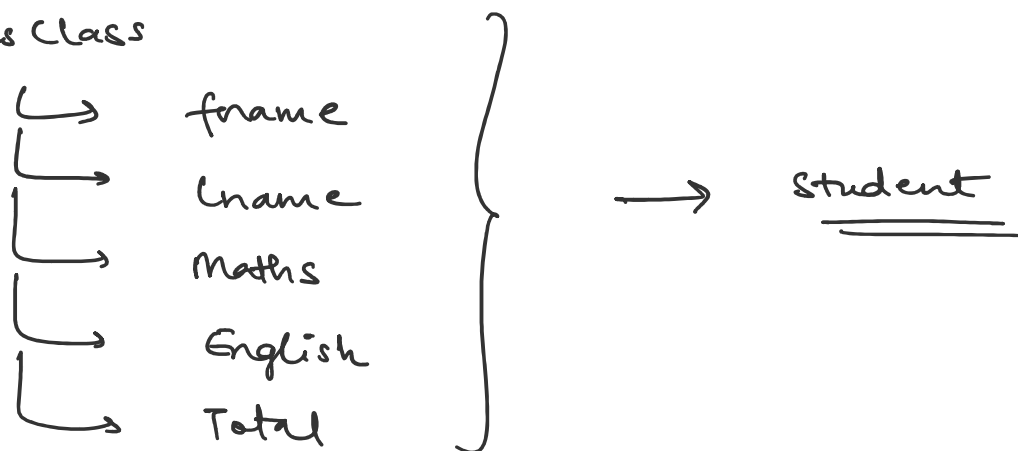


Student (Doe , John , 140 , 60 , 80)

Annotations below the arguments:

- ✓ above Doe, John, 140, 60, 80
- ↓ below Doe, John
- x below Doe, John
- String brace under Doe, John
- int brace under 140, 60, 80
- ✓ above 140, 60, 80
- Double checkmark (✓✓) to the right

Details Class



student (Details details) → Constructor
{

fname = details.fname ;

lname = details.lname ;

maths = details.maths

,
:
:

}

4 + parameters → complex object
↓
class inject that Bad design
↓
Good Design

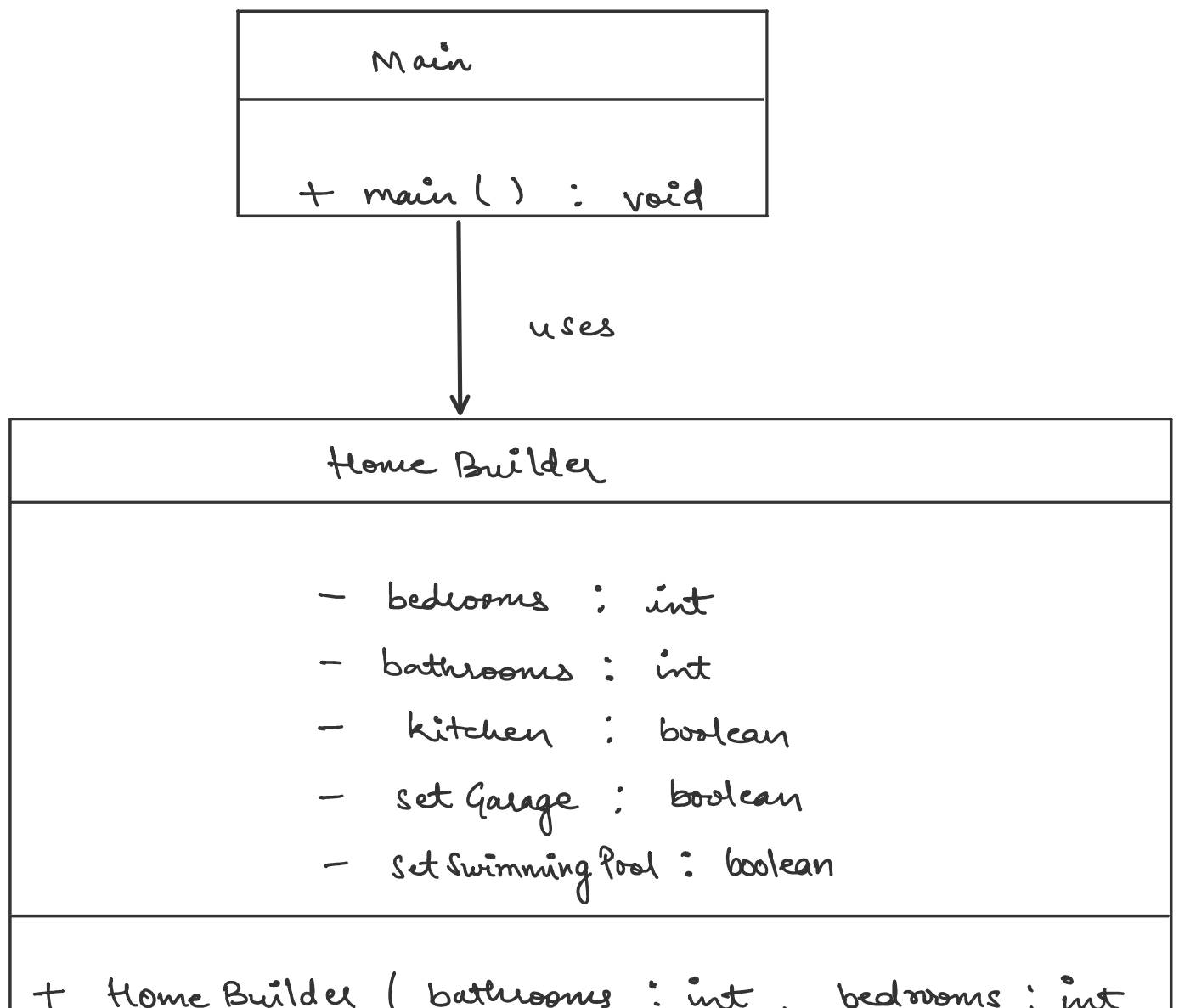
Home

}	↳	2 Bedrooms	→	Mandatory	✓
	↳	3 Washrooms	→	Mandatory	✓
	↳	1 swimming pool	→	optional	✓
	↳	1 Kitchen	→	Mandatory	✓
	↳	1 Garage	→	optional	✓
	↳	Interior	→	optional	✓



Apply Builder pattern

Class Diagram -



+ Home Builder (bathrooms : int , bedrooms : int ,
kitchen : boolean) :: void

+ set Garage (garage : boolean) :: Home Builder

+ setSwimmingPool (spool : boolean) :: Home Builder

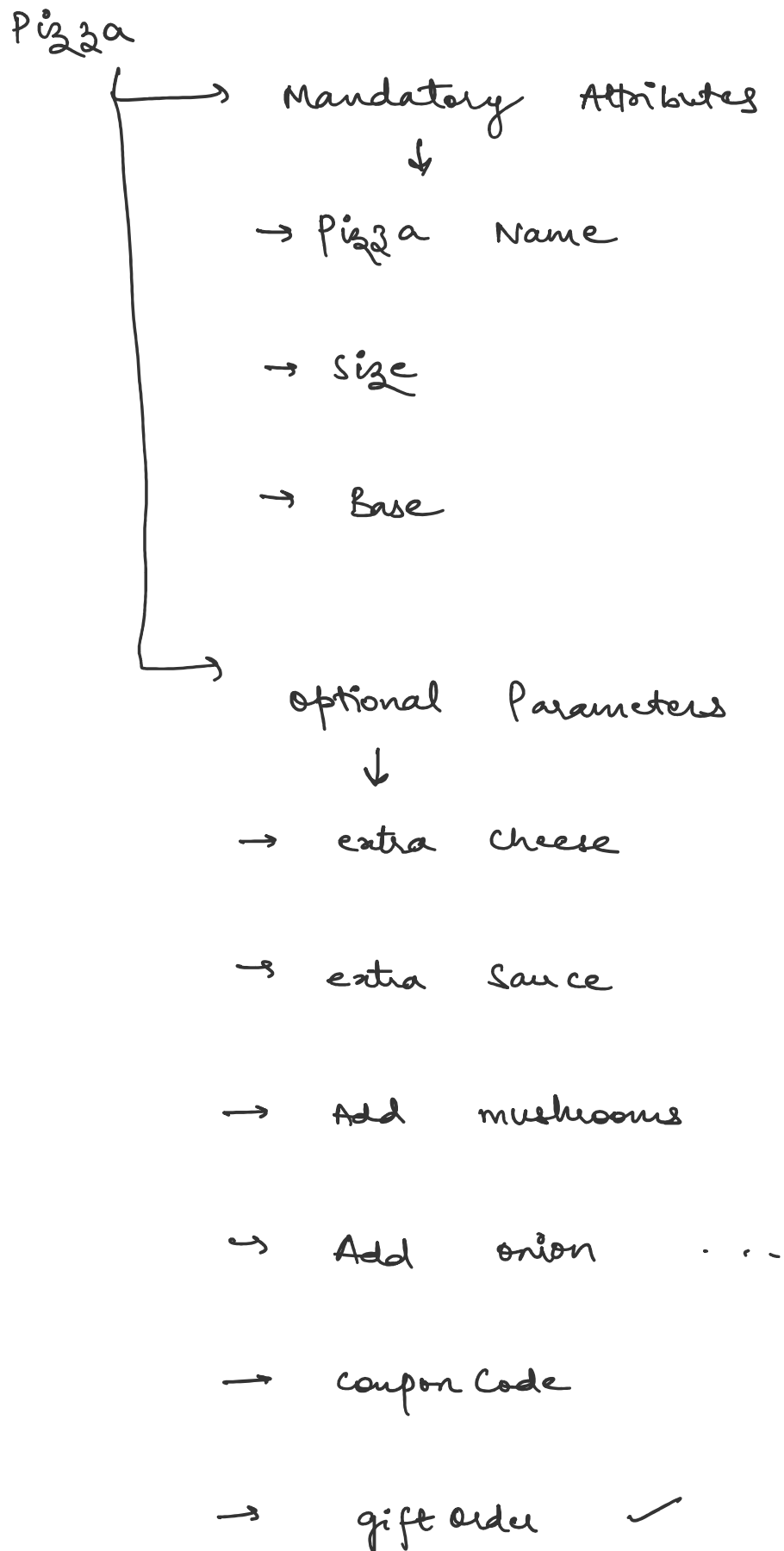
builds

Home

- bedrooms : int
- bathrooms : int
- kitchen : boolean
- set Garage : boolean
- setSwimmingPool : boolean

- Home (home builder : Home Builder)
- + toString () :: String

Another eg -



→ { special Instructions
→ { contactless delivery }

Main (Client)

↓ uses

Pizza Order Builder =

↓ builds

Pizza

Notes:

Links to draw diagrams:

- <https://mermaid.ai/>
- <https://app.diagrams.net/>

Builder Design Pattern

The **Builder Design Pattern** is a **Creational Design Pattern** used to **construct complex objects step by step**.

Instead of creating an object using a constructor with many parameters, a separate **Builder** class is responsible for constructing the object. The Builder collects all the required information, performs all necessary validations, and finally creates the object.

The Builder Pattern is especially useful when:

- The object has **many attributes**.
- Some attributes are **mandatory** while others are **optional**.
- The object should be created **only after all validations pass**.
- The object should be **immutable** after creation.

Why do we need Builder Pattern?

Consider a class having many attributes.

```
public class Batch {  
    private String name;  
    private BatchType type;  
    private int capacity;  
    private String instructor;  
    private LocalDate startDate;  
    private LocalDate endDate;  
    private List<Classroom> classes;  
    private boolean isRecorded;  
    private String description;  
}
```

Creating an object using constructors becomes difficult.

```
Batch batch = new Batch(  
    "Java Backend",  
    BatchType.ONLINE,  
    100,  
    "Rahul",  
    startDate,  
    endDate,  
    null,  
    true,  
    "Spring Boot Batch"  
);
```

Problems:

- **Hard to remember the order** of constructor parameters.
- **Multiple parameters may have the same data type**, making the code confusing.
- The code becomes **less readable**.
- Constructor becomes **difficult to maintain** as more attributes are added.

Example:

```
new Batch(  
    "Java",  
    "Rahul",  
    "Backend"  
);
```

Which String represents the **batch name**, **instructor**, or **description**?
The code becomes **unreadable**.

Real-life analogy

Imagine you're building a **house**.

Mandatory Parts

- Walls
- Roof
- Plumbing
- Wiring

Optional Parts

- Garage
- Swimming Pool
- Garden
- Interior Decoration

You cannot live in a house without the mandatory parts, but the optional features can be added if required.

Similarly, while creating an object:

- Some fields are **mandatory**.
- Some fields are **optional**.
- The object should be created **only after everything is properly configured**.

The Builder helps construct the object **step by step**.

Problems Solved by Builder Pattern

1. Too many constructor parameters

Instead of

```
new User(  
    "John",  
    25,  
    "Delhi",  
    "9876543210",  
    true,  
    false,  
    "Developer"  
)
```

We write

```
User user = new User.Builder("John")  
    .age(25)  
    .city("Delhi")  
    .phone("9876543210")  
    .isAdmin(true)
```

```
.build();
```

Much easier to read.

2. Telescoping Constructors

Suppose we have a User class.

```
public class User {  
    private String name;  
    private int age;  
    private String city;  
    private String phone;  
    private boolean admin;  
}
```

Instead of writing initialization logic in every constructor, each constructor delegates to another constructor.

```
public User(String name) {  
    this(name, 0);  
}
```

```
public User(String name, int age) {  
    this(name, age, null);  
}
```

```
public User(String name, int age, String city) {  
    this(name, age, city, null);  
}
```

```
public User(String name, int age, String city, String phone) {  
    this(name, age, city, phone, false);  
}
```

```
public User(String name, int age, String city, String phone,
boolean admin) {
    this.name = name;
    this.age = age;
    this.city = city;
    this.phone = phone;
    this.admin = admin;
}
```

What happens when we create an object?

User user = new User("John");

Execution flow:

User("John")

↓

User("John", 0)

↓

User("John", 0, null)

↓

User("John", 0, null, null)

↓

User("John", 0, null, null, false)

↓

Object Created

Each constructor internally calls another constructor. This is called a **Telescoping Constructor**. This results in a chain of constructor calls, making the code difficult to maintain and understand.

Problems with Telescoping Constructors

- Large number of overloaded constructors as attributes increase.

- **Difficult to read and maintain.**
- **Adding a new attribute** often requires creating another constructor and modifying existing ones.
- Constructors become **hard to understand**, especially when many parameters have the **same data type**.
- It is considered an **anti-pattern** for classes with many attributes.

3. Constructor Signature Collision

Suppose

Employee(String name, String department)

and

Employee(String department, String name)

Both have

(String, String)

Java cannot distinguish them. Cannot create both constructors.

4. Validation Before Object Creation

Sometimes objects should **not be created** unless all business rules are satisfied.

Example:

Before creating a **Batch**

- **No two batches should have the same name.**
- **Batch type must be provided.**
- **Capacity must be greater than zero.**
- **The list of classes should initially be null or empty.**
- **Start date must be before end date.**
- **Instructor cannot be null.**

If any validation fails,

`throw new IllegalArgumentException(...);`

or

`throw new IllegalStateException(...);`

The object should **never be created**. The **Builder** is the perfect place for performing these validations.

5. Immutable Objects

Builder is commonly used with **immutable classes**.

After the object is created,

- Its state **cannot be modified**.
- There are **no setters**.
- Fields are usually declared as **final**.

Example:

```
private final String name;  
private final int age;
```

Only getters are provided. No setters.

6. Restricted Values Using Enums

Sometimes an attribute should contain **only predefined values**.

Instead of

```
String batchSize;
```

use

```
BatchType batchSize;
```

```
public enum BatchType {  
    ONLINE,  
    OFFLINE,  
    HYBRID  
}
```

Now invalid values like "Online123", "Virtual" cannot be assigned. Enums provide **compile-time safety**.

When to Use Builder Pattern

Use the **Builder Pattern** when:

- You are constructing a **complex object**.
- The object has **many attributes** (typically **more than 4–5**), with some **mandatory** and some **optional**.
- **Constructors become difficult to read** because they have too many parameters.

- **Multiple constructor parameters have the same data type**, making the order of arguments confusing.
- You want to **avoid telescoping constructors** (multiple overloaded constructors).
- **Business validations** must be performed **before the object is created**.
- You want to create **immutable objects** (objects whose state cannot change after creation).
- You want a **Fluent API (method chaining)** for better readability.
- You want to **separate object construction logic** from the **business object** itself.
- You want to create **different representations/configurations of the same object** using the **same construction process**.

Example Scenarios

- Creating a **Batch** with validations.
- Creating a **PizzaOrder** with many optional toppings.
- Creating a **Home** with optional garage and swimming pool.
- Creating **API request objects**.
- Creating **configuration objects**.
- Creating **database connection objects**.
- Creating **HTTP request objects**.

When Should You Avoid Builder?

Avoid using the **Builder Pattern** when:

- The class has **only 2–3 attributes**.
- **Object creation is simple, straightforward** and does not involve complex construction logic.
- **No business validations** are required before creating the object.

- All attributes are **mandatory**, and a simple constructor is sufficient.
- There are **very few optional parameters**, making the Builder unnecessarily verbose.
- A **constructor (or static factory method)** provides a simpler and more readable solution.

In such cases, a constructor is sufficient, and introducing a Builder would add unnecessary complexity and boilerplate code.

Components of Builder Pattern

1. Product

The object that is being created.

Examples - Home, Batch, Car, PizzaOrder.

2. Builder

Responsible for constructing the product.

Responsibilities:

- Store values temporarily
- Validate input
- Support method chaining
- Create the final object

3. Client

Uses the Builder to construct the Product.

Example:

Home home =

`new Home.Builder(3,2)`

```
.garage(true)
.pool(true)
.build();
```

Builder Flow

Client



Builder



├— Set mandatory attributes

├— Set optional attributes

├— Perform validations

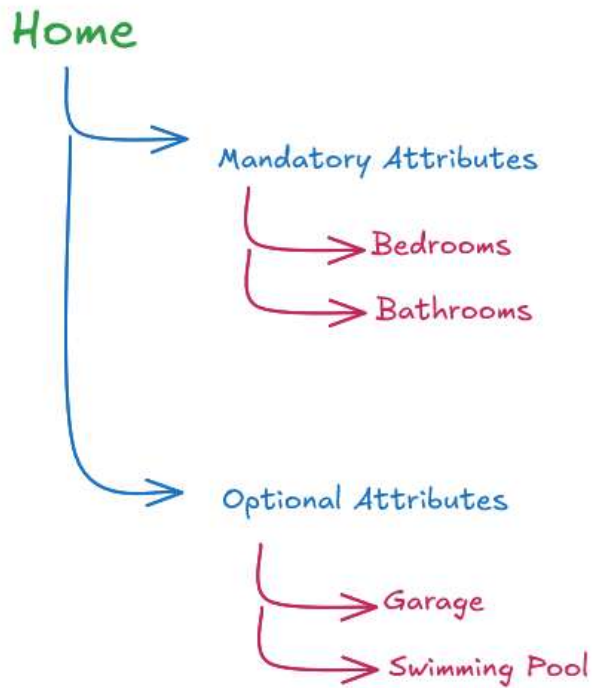


build()



Product

Implementation of Home Example



Step 1:

Identify a class with

- Many attributes
- Mandatory fields
- Optional fields

Step 2:

Make the constructor **private**.

```
private Home(HomeBuilder builder) {}
```

This ensures objects cannot be created directly.

Step 3:

Create a **static nested Builder class**.

`public static class` HomeBuilder

Step 4:

Builder constructor accepts **mandatory fields**.

`new HomeBuilder(3,2,true)`

Perform mandatory field validations here.

Step 5:

Provide methods for **optional fields**.

`garage(true)`
`pool(true)`

Each method returns
`return this;`

to enable **method chaining**.

Step 6:

Implement the **build()** method.

Responsibilities:

- Perform cross-field validations
- Create the Product
- Return the Product - `return new Home(this);`

// CLIENT -> uses -> HOMEBUILDER -> creates/builds -> HOME

```
public class Home {
```

// Builder is commonly used with immutable objects, make the fields final.

// mandatory

```
private final int bedrooms;  
private final int bathrooms;  
private final boolean kitchen;
```

// optional

```
private final boolean garage;  
private final boolean swimmingPool;
```

// Private constructor - can only be called by the Builder

```
private Home(HomeBuilder builder) {  
    this.bedrooms = builder.bedrooms;  
    this.bathrooms = builder.bathrooms;  
    this.kitchen = builder.kitchen;  
    this.garage = builder.garage;  
    this.swimmingPool = builder.swimmingPool;  
}
```

@Override

```
public String toString() {  
    return "Home : bedrooms=" + bedrooms +  
        ", bathrooms=" + bathrooms +  
        ", kitchen=" + kitchen +  
        ", garage=" + garage +  
        ", swimmingPool=" + swimmingPool;  
}
```

```

public static class HomeBuilder {
    // Builder Mandatory Fields Should Be final. These values
    // are supplied in the Builder constructor and never change.
    private final int bedrooms;
    private final int bathrooms;
    private final boolean kitchen;

    // Optional fields remain non-final because they are
    // configured later.
    // Optional fields (default values)
    private boolean garage = false;
    private boolean swimmingPool = false;

    // constructor for setting mandatory fields
    public HomeBuilder(int bedrooms, int bathrooms, boolean
kitchen) {
        // basic validation
        if (bedrooms <= 0) {
            throw new IllegalArgumentException("Bedrooms must
be greater than zero.");
        }

        if (bathrooms <= 0) {
            throw new IllegalArgumentException("Bathrooms must
be greater than zero.");
        }

        this.bedrooms = bedrooms;
        this.bathrooms = bathrooms;
        this.kitchen = kitchen;
    }

    // optional parameters
    public HomeBuilder swimmingPool(boolean swimmingPool)
{
        this.swimmingPool = swimmingPool;
        return this; // enabling method chaining - returning this
lets you call another method right after

```

```

    }

    public HomeBuilder garage(boolean garage) {
        this.garage = garage;
        return this;
    }

    // build method - creating home instance
    // Final validation and object creation
    public Home build() {
        // Example of cross-field validation
        if (swimmingPool && bedrooms < 3) {
            throw new IllegalStateException("A swimming pool is
allowed only for homes with at least 3 bedrooms.");
        }

        return new Home(this);
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Home home1 = new Home.HomeBuilder(3, 2, true)
            .swimmingPool(true)
            .garage(false)
            .build();

        System.out.println(home1);
        System.out.println("-----");

        Home home2 = new Home.HomeBuilder(4, 4, true)
            .swimmingPool(false)
            .garage(false)

```



```

        .build();

System.out.println(home2);
System.out.println("-----");

// Testcase to test validation
Home home3 = new Home.HomeBuilder(2, 1, true)
    .swimmingPool(true)
    .build();

System.out.println(home3);
    }
}

/* *

```

Output :

*Home : bedrooms=3, bathrooms=2, kitchen=true,
garage=false, swimmingPool=true*

*-----
Home : bedrooms=4, bathrooms=4, kitchen=true,
garage=false, swimmingPool=false*

*-----
Exception in thread "main" java.lang.IllegalStateException:
A swimming pool is allowed only for homes with at least 3
bedrooms.*

*at l03_design_patterns.creational.p05
_builder.HomeEg.Home\$HomeBuilder.build(Home.java:80)
at l03_design_patterns.creational.p05
_builder.HomeEg.Main.main(Main.java:25)*

** */*

Why Does the Builder Store Duplicate Fields?

Builder

bedrooms
bathrooms
garage

Home (Product)

bedrooms
bathrooms
garage

Although the fields look identical, they serve **different purposes**.

- The **Builder** acts as a **temporary container**, storing values while the object is being configured.
- When the build() method is called, these values are **validated** and **copied** into the **Product (Home)** object.
- After the object is created, the **Builder is discarded**, while the **Product stores the final state permanently**.

This separation:

- Supports **immutable objects**.
- Keeps the **construction logic** separate from the business object.
- Ensures only **fully initialized and valid objects** are created.

Fluent API (Method Chaining)

The Builder Pattern typically uses a **Fluent API**, where each Builder

method returns **this**.

`return this;`

This enables **method chaining**, making object creation more readable.

With Fluent API

```
Home home = new HomeBuilder(3, 2)
    .garage(true)
    .pool(false)
    .garden(true)
    .build();
```

Without Fluent API

```
HomeBuilder builder = new HomeBuilder(3, 2);

builder.garage(true);
builder.pool(false);
builder.garden(true);

Home home = builder.build();
```

Returning **this** allows multiple method calls to be chained together, resulting in **cleaner, more expressive, and more readable code**.

Types of Validations in Builder

Builder validations are typically performed at **three different stages**.

1. Constructor Validation

Used for validating **mandatory fields**.

Example:

```
public HomeBuilder(String name) {  
    if (name == null || name.isBlank()) {  
        throw new IllegalArgumentException("Name is required.");  
    }  
  
    this.name = name;  
}
```

Examples of constructor validations:

- Mandatory fields should not be null.
- Numeric values should be positive.
- Enum values should not be null.

2. Builder Method Validation

Used for validating **individual optional fields** as they are being set.

Example:

```
public HomeBuilder price(double price) {  
    if (price < 0) {  
        throw new IllegalArgumentException("Price cannot be  
negative.");  
    }  
}
```

```
this.price = price;  
return this;  
}
```

Examples:

- Price cannot be negative.
- Quantity cannot be less than zero.
- Discount percentage should be between **0–100**.

3. Cross-Field Validation (build())

Some business rules depend on **multiple fields** and therefore cannot be validated until **all values have been provided**.

These validations should be performed inside the **build()** method.

Example:

```
public Home build() {  
    if (startDate.isAfter(endDate)) {  
        throw new IllegalStateException("Start date must be before  
end date.");  
    }  
  
    return new Home(this);  
}
```

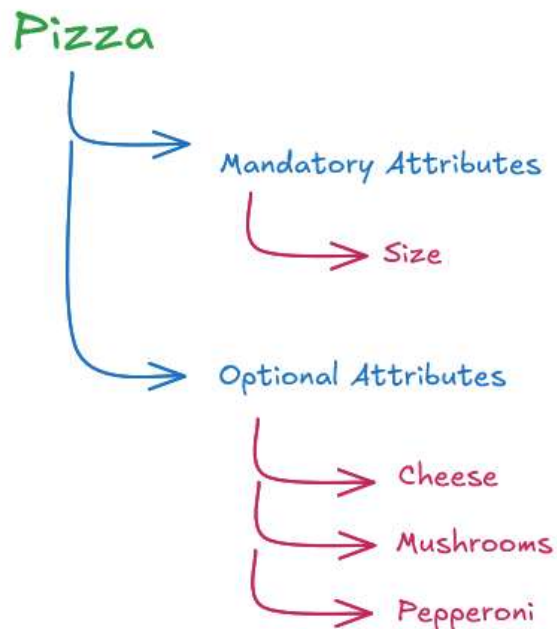
Examples of cross-field validations:

- **Start Date** must be before **End Date**.
- **Gift wrapping** is not available for **Cheese Burst** pizzas.
- **Capacity** should be greater than the number of enrolled students.
- **Free shipping** is available only if the order amount exceeds a

certain limit.

Only after **all validations pass** does the Builder create and return the final object.

Another Example



Step 1. PizzaSize Enum

```
public enum PizzaSize {  
    SMALL,  
    MEDIUM,  
    LARGE  
}
```

Step 2. CrustType Enum

```
public enum CrustType {  
    REGULAR,  
    THIN_CRUST,  
    CHEESE_BURST  
}
```

Step 3. PizzaOrder

```
public class PizzaOrder {  
    // Mandatory Fields  
    private final String pizzaName;  
    private final PizzaSize size;  
  
    // Optional Fields  
    private final CrustType crustType;  
    private final boolean extraCheese;  
    private final boolean extraSauce;  
    private final int oreganoSachets;  
    private final int chilliFlakesSachets;  
    private final String couponCode;  
    private final boolean giftWrap;  
    private final boolean contactlessDelivery;  
    private final String specialInstructions;  
  
    // Private constructor - can only be called by Builder  
    private PizzaOrder(Builder builder) {
```

```
this.pizzaName = builder.pizzaName;  
this.size = builder.size;  
this.crustType = builder.crustType;  
this.extraCheese = builder.extraCheese;  
this.extraSauce = builder.extraSauce;  
this.oreganoSachets = builder.oreganoSachets;  
this.chilliFlakesSachets = builder.chilliFlakesSachets;  
this.couponCode = builder.couponCode;  
this.giftWrap = builder.giftWrap;  
this.contactlessDelivery = builder.contactlessDelivery;  
this.specialInstructions = builder.specialInstructions;  
}
```

@Override

```
public String toString() {  
    return "PizzaOrder" +  
        "\n pizzaName=" + pizzaName + "\" +  
        ",\n size=" + size +  
        ",\n crustType=" + crustType +  
        ",\n extraCheese=" + extraCheese +  
        ",\n extraSauce=" + extraSauce +  
        ",\n oreganoSachets=" + oreganoSachets +  
        ",\n chilliFlakesSachets=" + chilliFlakesSachets +  
        ",\n couponCode=" + couponCode + "\" +  
        ",\n giftWrap=" + giftWrap +  
        ",\n contactlessDelivery=" + contactlessDelivery +  
        ",\n specialInstructions=" + specialInstructions + "\";  
}
```

```
public static class Builder {  
    // Mandatory  
    private final String pizzaName;  
    private final PizzaSize size;
```

```
    // Optional fields (default values)
```



```
private CrustType crustType = CrustType.REGULAR;
private boolean extraCheese = false;
private boolean extraSauce = false;
private int oreganoSachets = 1;
private int chilliFlakesSachets = 1;
private String couponCode = "";
private boolean giftWrap = false;
private boolean contactlessDelivery = false;
private String specialInstructions = "";
```

// Constructor Validation

```
public Builder(String pizzaName, PizzaSize size) {
    if (pizzaName == null || pizzaName.isBlank()) {
        throw new IllegalArgumentException("Pizza name is
required.");
    }

    if (size == null) {
        throw new IllegalArgumentException("Pizza size is
required.");
    }

    this.pizzaName = pizzaName;
    this.size = size;
}
```

// Optional field

```
public Builder crustType(CrustType crustType) {
    if (crustType != null) {
        this.crustType = crustType;
    }
    return this;
}
```

```
public Builder extraCheese(boolean value) {
    this.extraCheese = value;
}
```

```
        return this;
    }

    public Builder extraSauce(boolean value) {
        this.extraSauce = value;
        return this;
    }

    public Builder oreganoSachets(int count) {
        if (count < 0) {
            throw new IllegalArgumentException("Oregano
sachets cannot be negative.");
        }
        this.oreganoSachets = count;
        return this;
    }

    public Builder chilliFlakesSachets(int count) {
        if (count < 0) {
            throw new IllegalArgumentException("Chilli flakes
sachets cannot be negative.");
        }
        this.chilliFlakesSachets = count;
        return this;
    }

    public Builder couponCode(String couponCode) {
        if (couponCode == null) {
            return this;
        }

        couponCode = couponCode.trim();

        if (couponCode.length() > 20) {
            throw new IllegalArgumentException("Coupon code
cannot exceed 20 characters.");
        }
    }
}
```

```
    this.couponCode = couponCode;
    return this;
}
```

```
public Builder giftWrap(boolean value) {
    this.giftWrap = value;
    return this;
}
```

```
public Builder contactlessDelivery(boolean value) {
    this.contactlessDelivery = value;
    return this;
}
```

```
public Builder specialInstructions(String instructions) {
    if (instructions != null) {
        this.specialInstructions = instructions;
    }
    return this;
}
```

```
// Cross-field validation + object creation
public PizzaOrder build() {
    if (giftWrap && crustType ==
CrustType.CHEESE_BURST) {
        throw new IllegalStateException("Gift wrapping is not
available for Cheese Burst pizzas.");
    }

    if (extraSauce && size == PizzaSize.SMALL) {
        throw new IllegalStateException("Extra sauce is not
available for Small pizzas.");
    }

    if (oreganoSachets > 10) {
        throw new IllegalStateException("Maximum 10
oregano sachets are allowed.");
    }
}
```

```

    }

    if (contactlessDelivery && giftWrap) {
        throw new IllegalStateException("Gift wrapped orders
cannot be delivered contactlessly.");
    }

    if (extraCheese && crustType ==
CrustType.CHEESE_BURST) {
        throw new IllegalStateException("Extra cheese is
already included in Cheese Burst pizzas.");
    }

    return new PizzaOrder(this);
}
}
}

```

Step 4. Main

```

public class Main {
    public static void main(String[] args) {
        PizzaOrder order1 =
            new PizzaOrder.Builder(
                "Farmhouse",
                PizzaSize.LARGE)
                .crustType(CrustType.THIN_CRUST)
                .extraCheese(true)
                .extraSauce(true)
                .oreganoSachets(3)
                .chilliFlakesSachets(2)
                .couponCode("SAVE200")
                .giftWrap(true)
                .specialInstructions("Ring the bell once.")
    }
}

```

```
.build();
```

```
System.out.println(order1);  
System.out.println("-----");
```

```
PizzaOrder order2 =  
    new PizzaOrder.Builder(  
        "Veggie Delight",  
        PizzaSize.MEDIUM)  
        .extraCheese(true)  
        .contactlessDelivery(true)  
        .specialInstructions("Leave at the doorstep.")  
        .build();
```

```
System.out.println(order2);  
System.out.println("-----");
```

```
PizzaOrder order3 =  
    new PizzaOrder.Builder(  
        "Margherita",  
        PizzaSize.SMALL)  
        .build();
```

```
System.out.println(order3);  
System.out.println("-----");
```

// Invalid Order - Demonstrating Cross-field Validation

```
try {  
    PizzaOrder order4 =  
        new PizzaOrder.Builder(  
            "Cheese Burst",  
            PizzaSize.SMALL)  
            .crustType(CrustType.CHEESE_BURST)  
            .extraCheese(true)
```

```

        .build();

        System.out.println(order4);
    } catch (Exception e) {
        System.out.println("Exception : " + e.getMessage());
    }
}
}
}

```

```

/* *

```

Output :

```

PizzaOrder
pizzaName='Farmhouse',
size=LARGE,
crustType=THIN_CRUST,
extraCheese=true,
extraSauce=true,
oreganoSachets=3,
chilliFlakesSachets=2,
couponCode='SAVE200',
giftWrap=true,
contactlessDelivery=false,
specialInstructions='Ring the bell once.'

```

```

-----
PizzaOrder
pizzaName='Veggie Delight',
size=MEDIUM,
crustType=REGULAR,
extraCheese=true,
extraSauce=false,
oreganoSachets=1,
chilliFlakesSachets=1,
couponCode="",
giftWrap=false,
contactlessDelivery=true,
specialInstructions='Leave at the doorstep.'

```

```
-----  
PizzaOrder  
  pizzaName='Margherita',  
  size=SMALL,  
  crustType=REGULAR,  
  extraCheese=false,  
  extraSauce=false,  
  oreganoSachets=1,  
  chilliFlakesSachets=1,  
  couponCode="",  
  giftWrap=false,  
  contactlessDelivery=false,  
  specialInstructions=""  
-----
```

Exception : Extra cheese is already included in Cheese Burst pizzas.

* */

Advantages

- Improves **code readability**.
- Avoids **telescoping constructors**.
- Supports **immutable objects**.
- Centralizes **validation logic**.
- Supports **optional parameters**.
- Easy to **extend**.
- Provides a **Fluent API (method chaining)**.
- Separates **construction logic** from the Product.
- Prevents creation of **invalid objects**.

Disadvantages

- Introduces **additional classes**.
- Requires **more boilerplate code**.
- Slightly increases implementation complexity.
- Can be **overkill for simple objects**.

Interview Questions

Why not use constructors?

Too many parameters reduce readability.

Why constructor should be private?

So objects can only be created through the Builder after validation.

Why Builder is static?

It does not depend on an existing Product object.

If Builder were non-static

Home home = new Home();

would already create an object, defeating the purpose.

Why no setters?

Because Builder is generally used to create immutable objects.

Where should validations be written?

- Mandatory field validations → Builder constructor
- Optional field validations → Builder methods
- Cross-field/business rule validations → build()

Can Builder create mutable objects?

Yes.

But it is mostly used with immutable objects.

Aspect	Builder	Factory	Abstract Factory
Purpose	Step-by-step object creation	Single method to create object	Create families of related objects
Object Complexity	Complex objects with many attributes	Simple objects	Group of related objects
Control	Fine-grained control over construction process	Less control (one-shot creation)	Multiple factories for related products
Focus	How to build	What to build	Which family to build
Analogy	Building a house	Ordering a	Ordering a combo

	step by step	pizza	meal (pizza + drink + dessert)
--	--------------	-------	-----------------------------------